Linguaggi di Programmazione

Linguaggi liberi deterministici: DPDA.
Introduzione al parsing top-down e bottom-up.

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Linguaggi liberi deterministici: DPDA

PDA deterministico (DPDA) e linguaggio libero deterministico

PDA deterministico (DPDA)

Un PDA $N = (\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ è deterministico se valgono le seguenti due condizioni:

1. $\forall q \in Q,\ \forall Z \in \Gamma,\ \forall a \in \Sigma \cup \{\epsilon\}$, vale $|\delta(q, a, Z)| \leq 1$
2. $\forall q \in Q,\ \forall Z \in \Gamma$, se $\delta(q, \epsilon, Z) \neq \emptyset$, allora $\delta(q, a, Z) = \emptyset\ \forall a \in \Sigma$

che più brevemente possono essere riassunte come:

$$|\delta(q, a, Z)| + |\delta(q, \epsilon, Z)| \leq 1 \qquad \forall q \in Q,\ \forall Z \in \Gamma,\ \forall a \in \Sigma.$$

Linguaggio libero deterministico

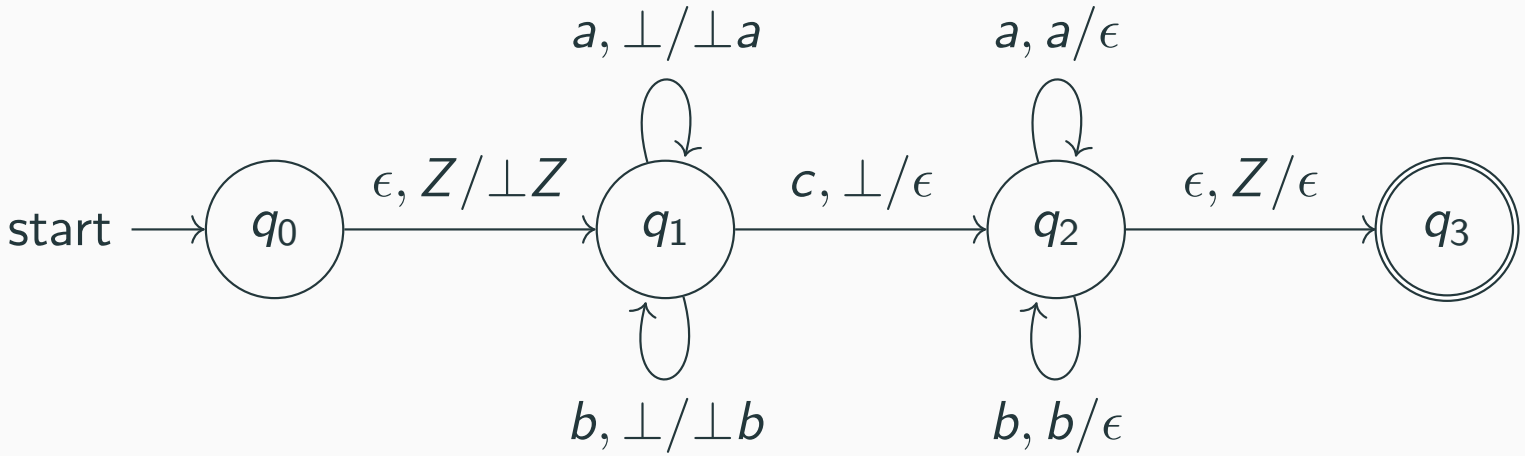
Un linguaggio è libero deterministico se è accettato **per stato finale** da un DPDA.

Linguaggi liberi deterministici e nondeterministici

Teorema
La classe dei linguaggi liberi deterministici è inclusa **propriamente** nella classe dei linguaggi liberi.

Infatti, il linguaggio $L_1 = \{ww^R | w \in \{a, b\}^*\}$ è libero (generato dalla grammatica $S \to aSa \mid bSb \mid \epsilon$), ma si può dimostrare che non esiste un DPDA che lo riconosce. Quindi L_1 è libero nondeterministico.

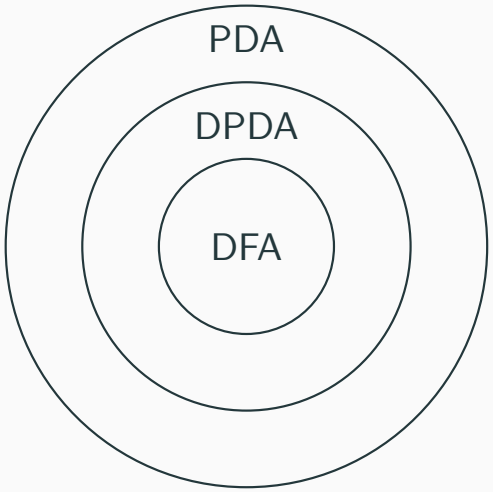
Invece, $L_2 = \{wcw^R | w \in \{a, b\}^*\}$ è libero deterministico, e un DPDA che lo riconosce per stato finale è

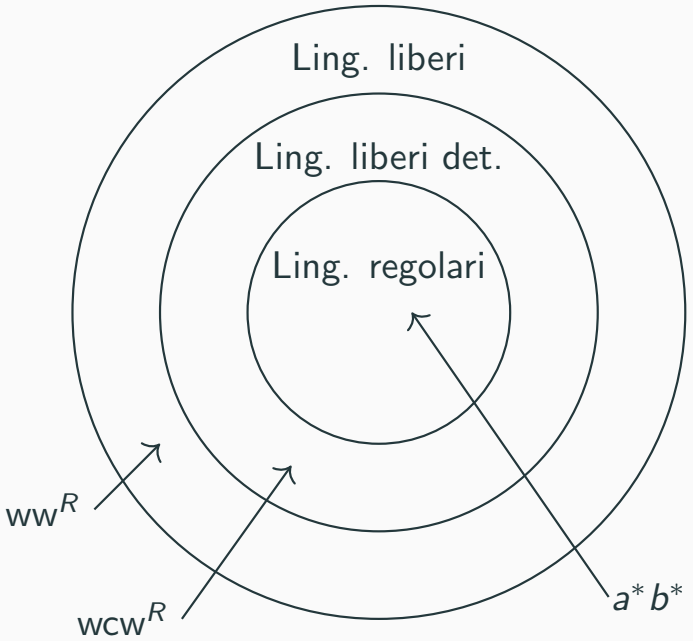


Classificazione

Proposizione: Se L è regolare, allora $\exists$ DPDA N tale che $L = L[N]$ (per stato finale)

Dimostrazione: Se L è regolare, allora esiste un DFA M tale che $L = L[M]$. A partire da M, posso costruire un DPDA N che si comporta come M senza mai manipolare lo stack, quindi $L = L[N]$.



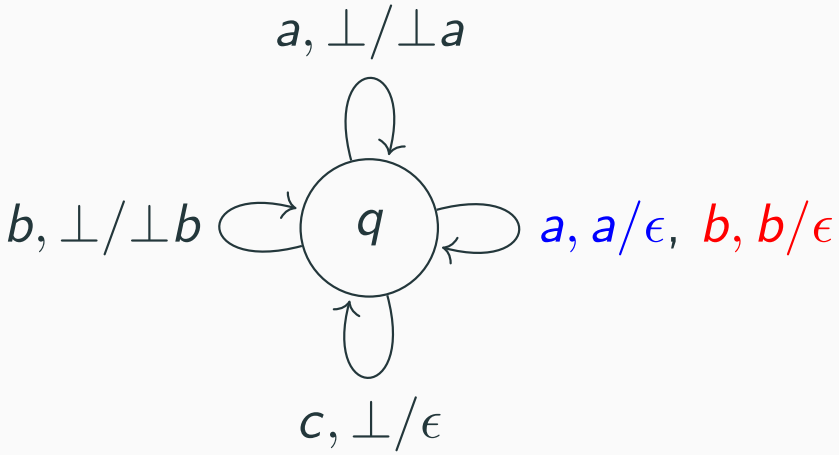


Linguaggi liberi deterministici riconosciuti per pila vuota

Un linguaggio libero deterministico L è riconosciuto da un DPDA **per pila vuota** se e solo se L è **prefix free**:

$$\nexists x, y \in L \text{ tali che } x \text{ è prefisso di } y$$

Ad esempio, $L_2 = \{wcw^R | w \in \{a, b\}^*\}$ è prefix-free. Quindi può essere riconosciuto da un DPDA anche per pila vuota:



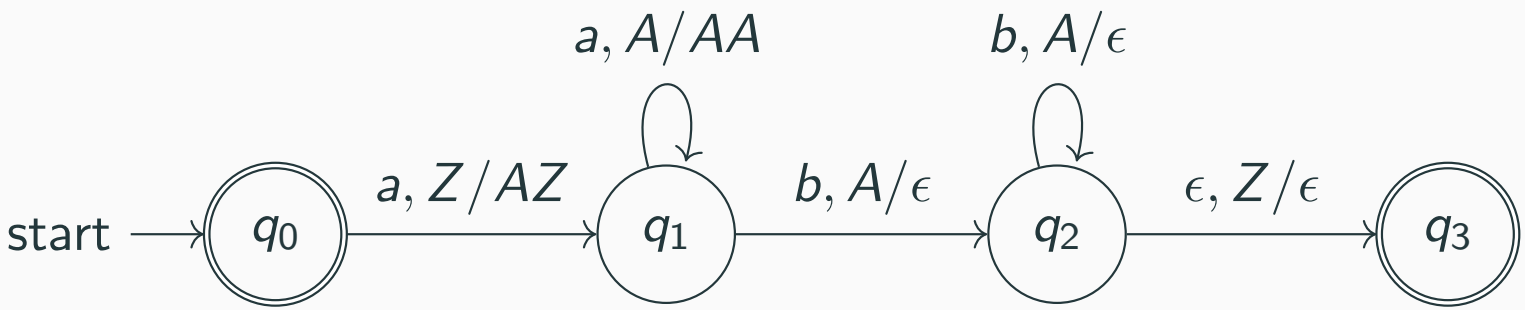
DPA per pila vuota $\neq$ DPDA per stato finale

Quindi la classe dei linguaggi liberi deterministici riconosciuta dai DPDA per pila vuota è una sottoclasse propria dei linguaggi liberi deterministici, ovvero dei linguaggi riconosciuti dai DPDA per stato finale.
Vale che:

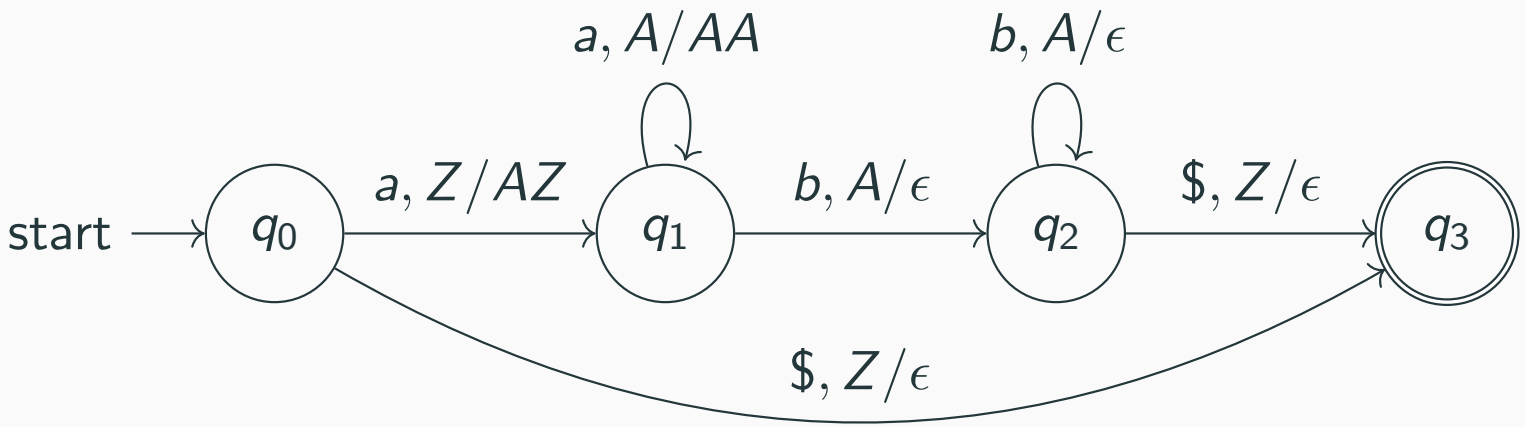
- Se L libero deterministico **è prefix-free**, allora può essere riconosciuto da un DPDA per pila vuota.
- Se L libero deterministico **non è prefix-free**, allora L non può essere riconosciuto da un DPDA per pila vuota.
- Se L è libero deterministico, allora $L\$ = \{w\$ | w \in L\}$ **è prefix-free**, e quindi $L\$$ può essere riconosciuto da un DPDA per pila vuota.
  - Vedremo che useremo sempre il trucco di aggiungere l'endmarker \$ per far sì che il linguaggio sia prefix-free, perché i parser che andremo a costruire saranno essenzialmente dei PDA che riconoscono per pila vuota, il cui funzionamento è reso deterministico con l'ausilio di particolari strutture, dette tabelle di parsing.

Esempio

$L_1 = \{a^n b^n | n \geq 0\}$ non è prefix-free perchè $\epsilon \in L_1$ ed ϵ è prefisso di ab.
Questo DPDA lo riconosce per stato finale

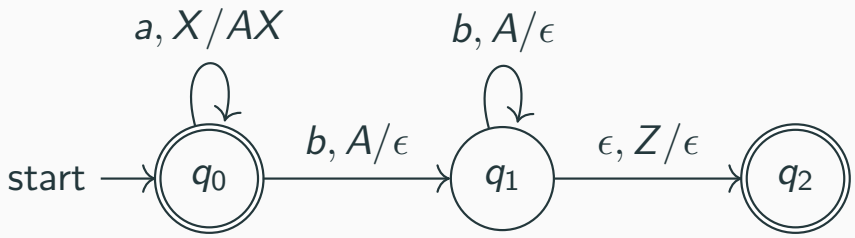


Questo DPDA riconosce $L_1\$$ per pila vuota (ed anche per stato finale)

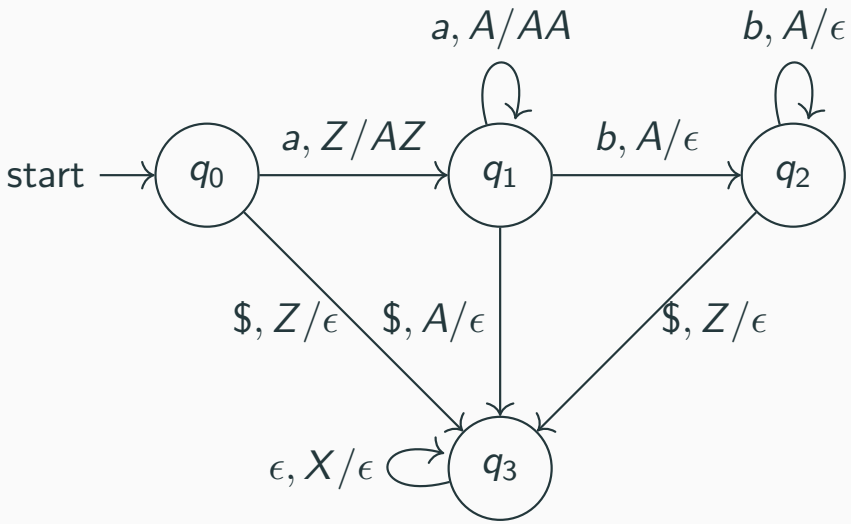


Esempio

$L_2 = \{a^n \mid n \geq 0\} \cup \{a^n b^n \mid n \geq 1\}$ non è prefix-free (perché, ad esempio, ϵ è prefisso di qualunque altra stringa). Questo DPDA lo riconosce per stato finale:

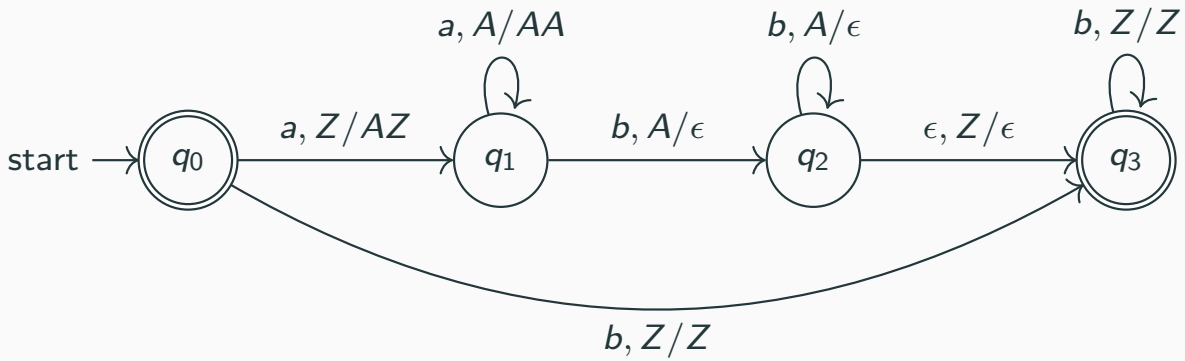


Questo DPDA riconosce $L_2\$$ per pila vuota:

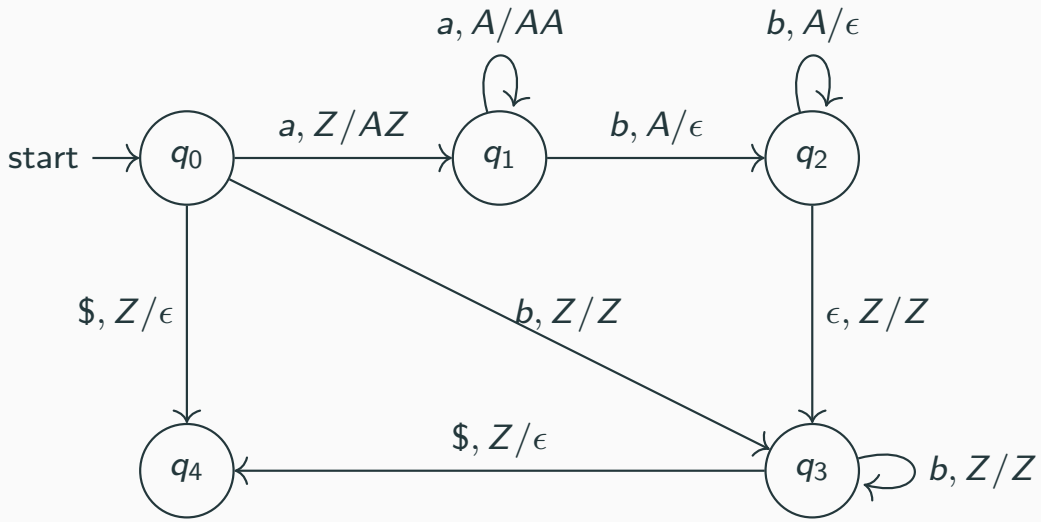


Esempio

$L_3 = \{a^n b^m \mid n \leq m\}$ non è prefix-free (perché, ad esempio, ab è prefisso di abb).
Questo DPDA lo riconosce per stato finale:



Questo DPDA riconosce $L_3\$$ per pila vuota:



Esercizi

1. $L_4 = \{a^n b^m \mid n \geq m \geq 0\}$ non è prefix-free (perché?). Costruire un DPDA che riconosca L_4 per stato finale, e un DPDA che riconosca $L_4\$$ per pila vuota.
2. $L_5 = \{a^n b^m a^n \mid n, m \geq 1\}$ è prefix-free (perché?). Costruire un DPDA che riconosca L_5 per pila vuota.
3. Costruire un singolo DPDA che riconosca il linguaggio $L_6 = \{a^n b^n \mid n \geq 1\}$ sia per pila vuota che per stato finale. Sarebbe possibile risolvere questo problema per $L_7 = \{a^n b^n \mid n \geq 0\}$? Motivare la risposta.
4. Costruire un DPDA che riconosca il linguaggio $L_8 = \{x = y \mid x \in \{1,2\}^*, y \in \{5\}^*, |x|_1 + 2 \cdot |x|_2 = 5 \cdot |y|_5\}$, dove $|z|_a$ indica quanti simboli a occorrono nella stringa z. (Si tratta di un convertitore da monete da 1 o 2 euro in banconote da 5 euro).

Proprietà dei linguaggi liberi deterministici: non ambigui!

Proposizione
Se L è libero deterministico (cioè riconosciuto da un DPDA per stato finale), allora L è generabile da una grammatica libera **non ambigua**.

Quindi, i linguaggi liberi deterministici non sono ambigui, perché ammettono almeno una grammatica non ambigua che li genera!

N.B.: i linguaggi di programmazione sono definiti attraverso grammatiche libere non ambigue che generano linguaggi liberi deterministici.

Proprietà di chiusura dei linguaggi liberi deterministici

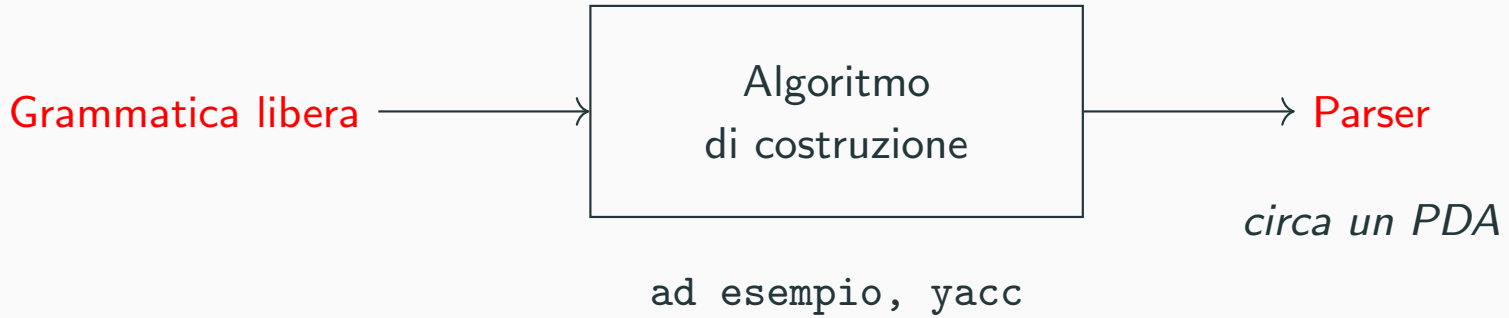
- Sono chiusi per complementazione, cioè se esiste un DPDA N tale che $L = L[N]$, allora esiste un DPDA N' tale che $\overline{L} = L[N']$, dove $\overline{L} = \Sigma^* \setminus L$.
  - **Idea della prova**: È possibile trasformare il DPDA N in uno equivalente che può leggere ogni possibile input (cioè tale che $|\delta(q, a, Z)| + |\delta(q, \epsilon, Z)| = 1 \quad \forall q \in Q,\ \forall Z \in \Gamma,\ \forall a \in \Sigma$) e tale che le sue transizioni ϵ non entrino mai in uno stato finale. Poi il DPDA N' si ottiene da questo N aumentato (con una costruzione particolare, che ometto) essenzialmente scambiando finali e non finali (come fatto per i DFA).
- L'intersezione $L_1 \cap L_2$ tra un libero deterministico L_1 e un regolare L_2 è un linguaggio libero deterministico. (La prova che abbiamo fatto nel caso dei liberi funziona anche per i liberi deterministici.)

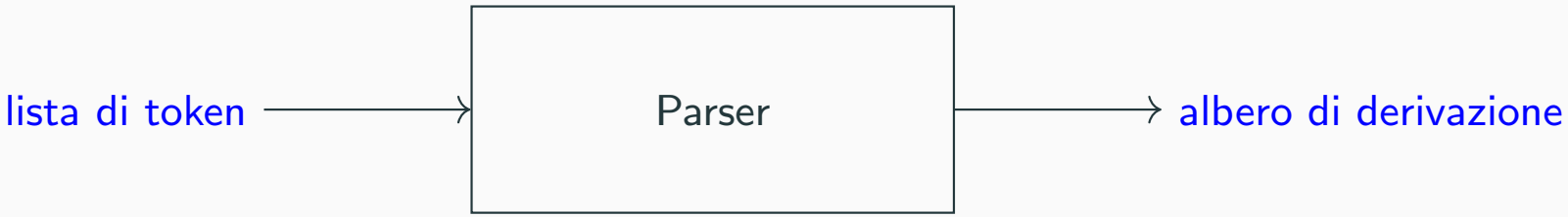
Proprietà di (non) chiusura dei linguaggi liberi deterministici

- Non sono chiusi per intersezione
  - $L_1 = \{a^n b^n c^m | n, m \geq 0\}$ è libero deterministico
  - $L_2 = \{a^m b^n c^n | n, m \geq 0\}$ è libero deterministico
  - $L_1 \cap L_2 = \{a^n b^n c^n | n, m \geq 0\}$ NON è libero!
- Non sono chiusi per unione
  - Se, per assurdo, lo fossero, allora $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ e quindi risulterebbero chiusi per intersezione, il che è impossibile!
- Non sono chiusi per concatenazione, né per stella di Kleene (controesempi non semplici)

Introduzione agli analizzatori sintattici (parser)

Analizzatori sintattici (Parser)





Analizzatori sintattici (Parser)

I **Parser** possono essere

- Nondeterministici: se, durante la ricerca di una derivazione, si scopre che una scelta è improduttiva e non porta a riconoscere l'input, il parser torna indietro (backtracking), disfa parte della derivazione appena costruita, e sceglie un'altra produzione, tornando eventualmente anche a rileggere (parte del)l'input
- Deterministici: leggono l'input una volta sola; ogni loro decisione è definitiva. (Complessità lineare nella lunghezza dell'input.)

Entrambi cercano di sfruttare informazioni dell'input per guidare la ricerca della derivazione.

Analizzatori sintattici (Parser)

I **Parser** possono essere

- Top-down: ricostruiscono una derivazione **leftmost** per w a partire dal simbolo iniziale S (all'inizio sulla pila).
- Bottom-up: ricostruiscono una derivazione **rightmost (a rovescio)** a partire dalla stringa w, cercando di ridurla al simbolo iniziale S (alla fine sulla pila).

Entrambi cercano di sfruttare informazioni dell'input per guidare la ricerca della derivazione.

Vedremo parser top-down deterministici (ottenuti a partire da grammatiche LL(k), in particolare LL(1)), e parser bottom-up deterministici (ottenuti a partire da grammatiche LR(k), in particolare LR(0), SLR(1), LR(1) e LALR(1)).

Introduzione al top-down parsing

Introduzione al Top-down Parsing

Data $G = (NT, T, S, R)$ libera, costruiamo il PDA con un solo stato $M = (T, \{q\}, T \cup NT, \delta, q, S, \emptyset)$ che riconosce per pila vuota, dove δ è definita come segue:

- $(q, \beta) \in \delta(q, \epsilon, A)$ se $A \to \beta \in R$ (espandi)
- $(q, \epsilon) \in \delta(q, a, a)$ per ogni $a \in T$ (consuma)

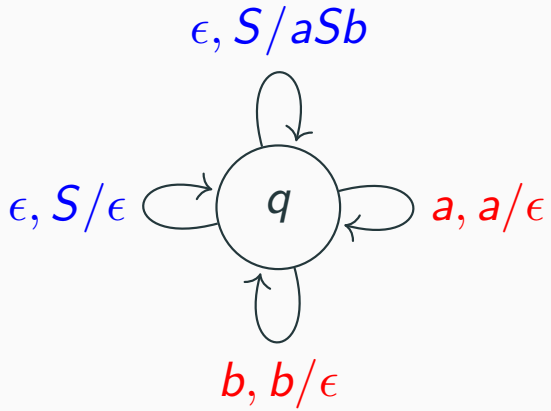
tale che $L(G) = P[M]$.

N.B.: le transizioni di tipo espandi generano nondeterminismo!

Questo tipo di automa è un basico parser top-down nondeterministico.

Esempio

Data la grammatica $G = S \to aSb \mid \epsilon$, il suo PDA top-down N è:



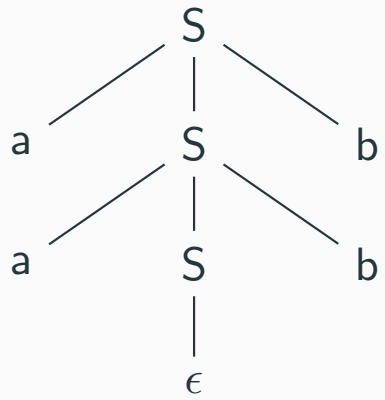
Consideriamo la seguente computazione del PDA N con input $aabb$:

$(q, aabb, S) \vdash_N (q, aabb, aSb) \vdash_N (q, abb, Sb) \vdash_N (q, abb, aSbb) \vdash_N$
$(q, bb, Sbb) \vdash_N (q, bb, bb) \vdash_N (q, b, b) \vdash_N (q, \epsilon, \epsilon)$

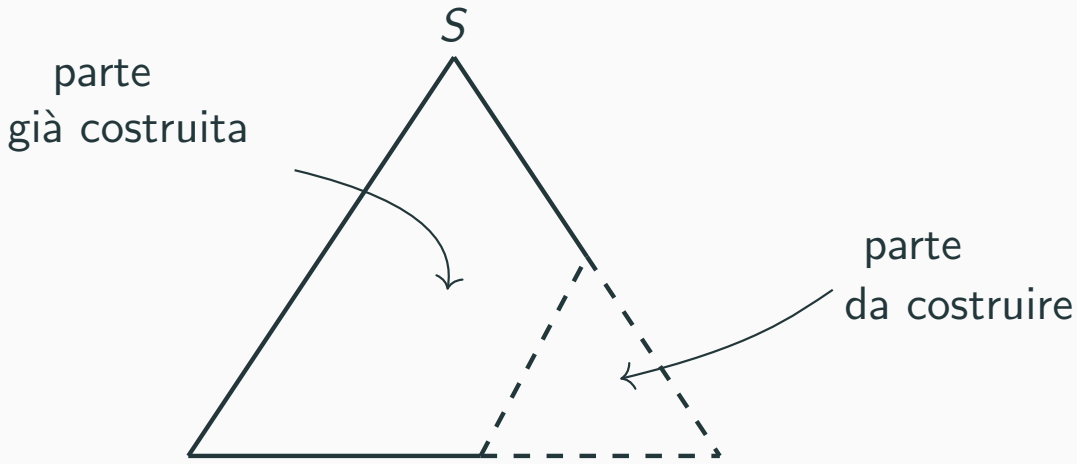
Ogni passo di tipo espandi produce un passo della derivazione **leftmost**, dalla quale si ricava un albero di derivazione. La derivazione leftmost è $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$.

Esempio

L'albero di derivazione che viene generato (dall'alto verso il basso, ovvero top-down) è:



In generale, si parte dalla radice S e si procede in modo top-down:



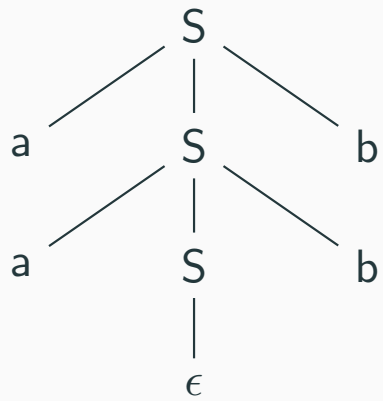
Nondeterminismo

A volte, il nondeterminismo nella scelta della produzione da espandere può essere risolto scegliendo la produzione in base al prossimo simbolo in lettura (detto look-ahead)

- se leggo "a" espando $S \to aSb$
- se leggo "b" espando $S \to \epsilon$

input	stack
$\underline{a}abb\$$	S
	$\underline{a}Sb$
$\underline{a}bb\$$	Sb
	$\underline{a}Sbb$
$\underline{b}b\$$	Sbb
	$\underline{b}b$
$b\$$	b
$\$$	ϵ

Albero di derivazione



Grammatiche LL(k)

Vedremo che la grammatica $G_1 = S \to aSb \mid \epsilon$ è di classe **LL(1)** nel senso che consente di costruire un parser deterministico

- **L**eft-to-right: modo in cui viene letto l'input,
- **L**eft-derivation: genera una derivazione leftmost,
- **1**: guardando un solo simbolo di look-ahead.

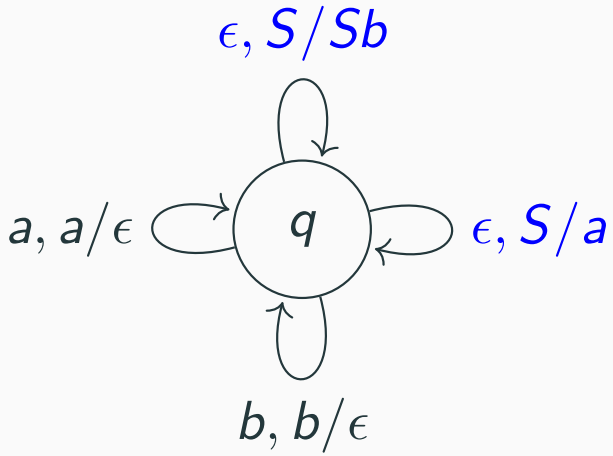
Vedremo che la grammatica $G_2 = S \to aSb \mid ab$ è di classe **LL(2)**: in questo caso devo guardare 2 simboli di look-ahead per avere determinismo:

- se leggo "aa" allora espando $S \to aSb$
- se leggo "ab" allora espando $S \to ab$

In generale, studieremo grammatiche **LL(k)** che guardano fino a k simboli in avanti.

Non tutte le grammatiche sono adatte per top-down parsing

La grammatica $G = S \to Sb \mid a$ che genera il linguaggio $L(G) = ab^*$ è **left recursive** (ricorsiva sinistra):



In questo caso c'è la possibilità di divergenza:
$(q, ab, S) \vdash_N (a, ab, Sb) \vdash_N (q, ab, Sbb) \vdash_N \ldots$

- non consumo mai l'input,
- continuo ad espandere S e aggiungere caratteri sulla pila.

Bisogna manipolare la grammatica G per ottenere una equivalente G' senza ricorsione sinistra.

Eliminare la ricorsione sinistra

Vedremo una tecnica generale in seguito. Per questo esempio, vale che

$$\begin{aligned} G' = & S \to aA \\ & A \to bA \mid \epsilon \end{aligned}$$

con $L(G') = L(G) = ab^*$, ma G' è adatta al top-down parsing:

- Se leggo "a" e sulla pila c'è S, espando $S \to aA$
- Se leggo "b" e sulla pila c'è A, espando $A \to bA$
- Se leggo "\$" e sulla pila c'è A, espando $A \to \epsilon$

Vedremo che G' è di classe LL(1).

N.B.: l'endmarker \$ mi serve per avere determinismo, altrimenti non saprei quando utilizzare la produzione $A \to \epsilon$

Ridurre il nondeterminismo usando fattorizzazione

Assumiamo di avere questa (porzione) di grammatica

$$\begin{aligned} S &\rightarrow abB \mid acC \\ B &\rightarrow \ldots \\ C &\rightarrow \ldots \end{aligned}$$

Se leggo "a" e sulla pila c'è S, come espando S?

- o leggo due simboli ($ab \Rightarrow S \rightarrow abB$ e $ac \Rightarrow S \rightarrow acC$)
- oppure **fattorizzo**: rimuovo il nondeterminismo, aggiungendo un nuovo nonterminale E, che è prefissato dalla parte comune alle due produzioni (il simbolo a), e le cui produzioni contengono la continuazione delle produzioni originali:

$$S \rightarrow aE$$

$$E \rightarrow bB \mid cC$$

$$\ldots$$

Ora basta un solo simbolo di look-ahead per risolvere il nondeterminismo.

Introduzione al bottom-up parsing

Introduzione al Bottom-up Parsing

Data $G = (NT, T, S, R)$ libera, costruiamo il PDA con un solo stato $M = (T, \{q\}, T \cup NT \cup \{Z\}, \delta, q, Z, \emptyset)$, che riconosce $L(G) \cdot \$$ per pila vuota, dove la funzione di transizione δ è definita come segue:

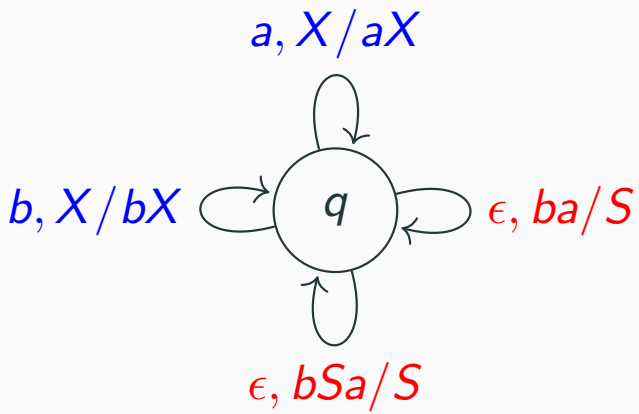
- $(q, aX) \in \delta(q, a, X) \quad \forall a \in T, \forall X \in T \cup NT \cup \{Z\}$ (shift)
- $(q, A) \in \delta(q, \epsilon, \alpha^R) \quad$ se $A \to \alpha \in R$ (reduce)
- $(q, \epsilon) \in \delta(q, \$, SZ) \quad$ (accept)

Notare che:

- Per le mosse di tipo **reduce**, generalizziamo i PDA per consumare sulla pila una stringa (α^R, ovvero il reverse del corpo della produzione) anziché solo il top.
- Per la mossa **accept**, \$ è il simbolo di fine input e S deve essere in cima alla pila, per poter fare la mossa finale.

Questo tipo di automa è un basico parser bottom-up (detto anche parser shift-reduce) nondeterministico.

Esempio $G = S \to aSb \mid ab$

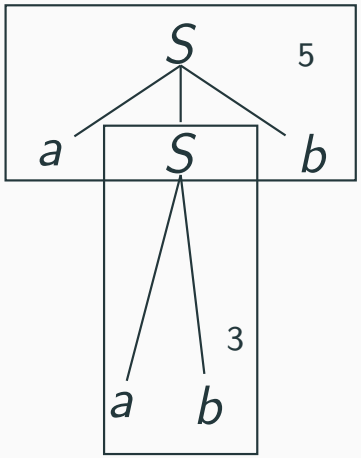


Step	Stack	Input	azione
0	Z	$aabb\$$	shift
1	Za	$abb\$$	shift
2	Zaa	$bb\$$	shift
3	$Zaab$	$b\$$	reduce $S \to ab$
4	ZaS	$b\$$	shift
5	$ZaSb$	$\$$	reduce $S \to aSb$
6	ZS	$\$$	accept
7	ϵ	ϵ	(fine)

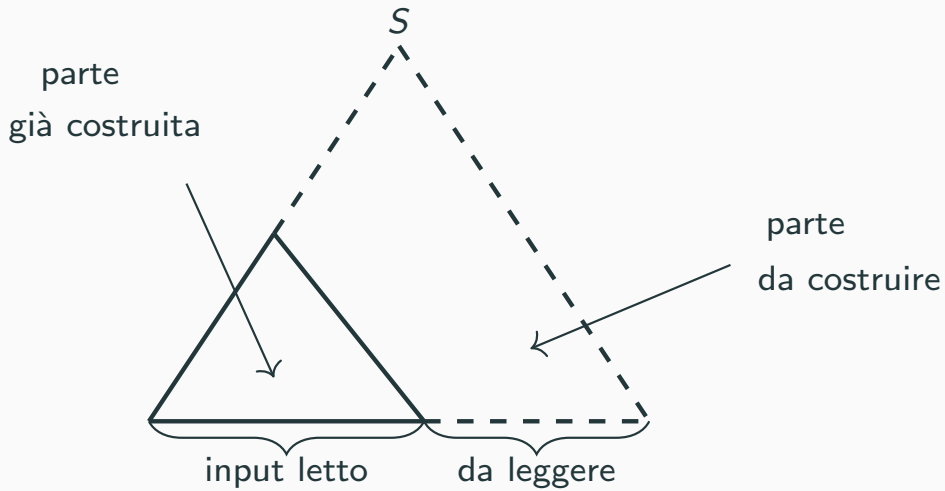
Esempio $G = S \to aSb \mid ab$

Leggendo, nella tabella della slide precedente, lo stack (che cresce verso destra) affiancato all'input, vediamo che, procedendo dall'alto verso il basso, si ottiene una derivazione **rightmost** a rovescio: $aabb \Leftarrow aSb \Leftarrow S$

Si vede che l'albero viene costruito bootm-up: prima la parte in basso (passo 3), poi la parte in alto (passo 5):



In generale, si inizia facendo delle shift e poi man mano si genera l'albero partendo dall'angolo in basso a sinistra:



Nondeterminismo

C'è molto nondetermnismo:

- Conflitti shift-reduce: Ad esempio al passo 3
  - 3) $Zaab \quad b\$$ potrei fare shift (invece di reduce), ottenendo
  - $4')$ $Zaabb \quad \$,$ ma è un percorso infruttuoso.
- Conflitti reduce-reduce, i.e., più reduce potrebbero essere possibili (non ci sono in questo semplice esempio).

Quindi, per ottenere un parser deterministico, servono informazioni aggiuntive per risolvere i conflitti:

- strutture particolari di supporto alla decisione (DFA dei prefissi viabili che, vedremo, ci permetterà di costruire una tabella di parsing che in alcuni casi potrà indicare univocamente l'azione da eseguire)
- look-ahead, che in alcuni casi potrà aiutare nel risolvere i conflitti.

Esempio più complesso: grammatica delle espressioni

Vediamo un esempio più complesso: la grammatica delle espressioni aritmetiche.

$$E \to T + E \mid T$$
$$T \to T * A \mid A$$
$$A \to a \mid b \mid (E)$$

Il parser bottom-up nondeterministico è descritto in forma tabellare come segue:

Produzione	Transizione	Azione
	$\delta(q, a, X) = (q, aX)$	shift
$E \to T + E$	$\delta(q, \epsilon, E + T) = (q, E)$	reduce R_1
$E \to T$	$\delta(q, \epsilon, T) = (q, E)$	reduce R_2
$T \to T * A$	$\delta(q, \epsilon, A * T) = (q, T)$	reduce R_3
$T \to A$	$\delta(q, \epsilon, A) = (q, T)$	reduce R_4
$A \to (E)$	$\delta(q, \epsilon,)E() = (q, A)$	reduce R_5
$A \to a$	$\delta(q, \epsilon, a) = (q, A)$	reduce R_6
$A \to b$	$\delta(q, \epsilon, b) = (q, A)$	reduce R_7
	$\delta(q, \$, EZ) = (q, \epsilon)$	accept

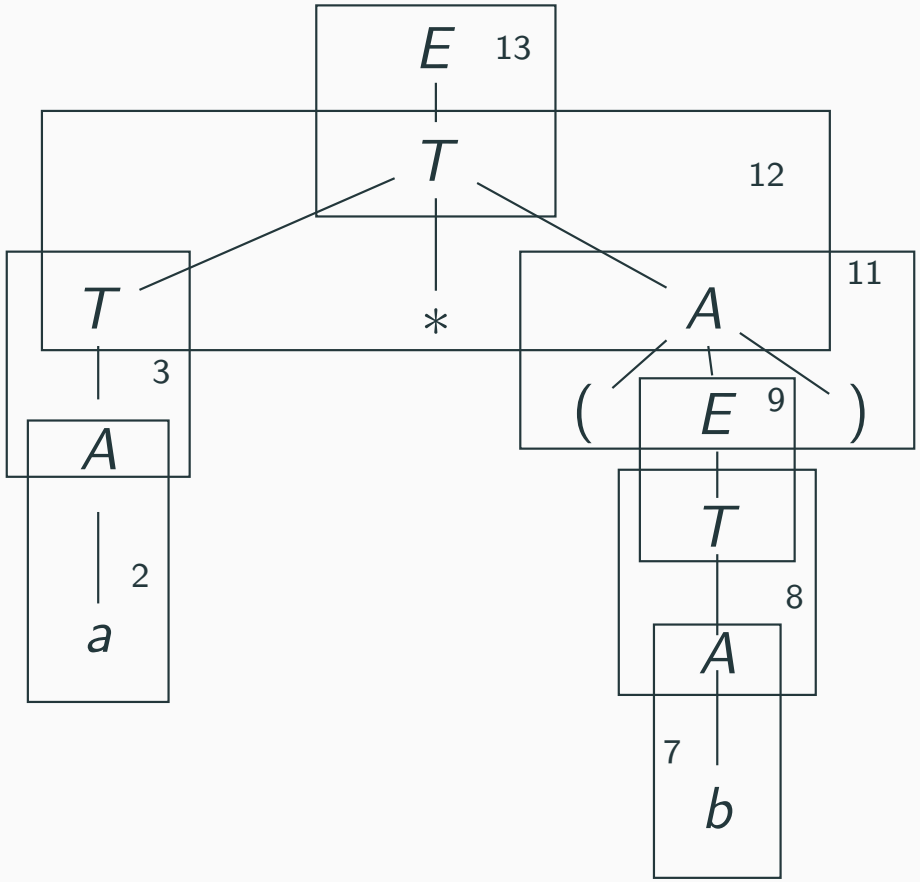
Esempio: grammatica delle espressioni artimetiche

Una derivazione canonica destra a rovescio è la seguente:
$a * (b) \Leftarrow A * (b) \Leftarrow T * (b) \Leftarrow T * (A) \Leftarrow T * (T) \Leftarrow T * (E) \Leftarrow T * A \Leftarrow T \Leftarrow E$

Step	Stack	Input	azione
0	Z	$a * (b)\$$	shift
1	$Z\underline{a}$	$*(b)\$$	reduce R_6
2	$Z\underline{A}$	$*(b)\$$	reduce R_4
3	ZT	$*(b)\$$	shift
4	$ZT*$	$(b)\$$	shift
5	$ZT*($	$b)\$$	shift
6	$ZT*(\underline{b}$	$)\$$	reduce R_7
7	$ZT*(\underline{A}$	$)\$$	reduce R_4
8	$ZT*(\underline{T}$	$)\$$	reduce R_2
9	$ZT*(E$	$)\$$	shift
10	$ZT*\underline{(E)}$	$\$$	reduce R_5
11	$Z\underline{T*A}$	$\$$	reduce R_3
12	$Z\underline{T}$	$\$$	reduce R_2
13	ZE	$\$$	accept
14	ϵ	ϵ	(fine)

Esempio: gramatica delle espressioni aritmetiche

Costruzione bottom-up dell'albero di derivazione



Nondeterminismo

Il parser è nondeterministico perché

- chi ha precedenza tra shift e reduce?
  - 1) $Za \quad *(b)\$$ potrei fare shift, e ottenere
  - 2′) $Za* \quad (b)\$$ scelta infruttuosa! Qui reduce è meglio di shift.
- Altro esempio su: chi ha precedenza tra shift e reduce?
  - 3) $ZT \quad *(b)\$$ potrei fare reduce R_2, e ottenere
  - 4′) $ZE \quad *(b)\$$ Scelta infruttuosa! Qui shift è meglio di reduce.
- chi ha precedenza tra due diverse reduce?
  - 11) $ZT*A \quad \$$ se faccio reduce R_3, ottengo
  - 12) $ZT \quad \$$ oppure, se faccio reduce R_4, ottengo
  - 12′) $ZT*T \quad \$$ Scelta infruttuosa! Qui reduce R_3 è meglio di reduce R_4.

È buona norma scegliere in modo tale che ciò che si trova sulla pila sia un prefisso (a rovescio) di una parte destra di una produzione della grammatica. Ad esempio, qui scelgo 2):

- 2′) $Za*$ non c'è nessuna produzione il cui reverse comincia con $a*$
- 2) $Z\underline{A}$ c'è una produzione il cui reverse comincia con A ($T \to A$)

Come ottenere parser bottom-up deterministici?

Vedremo che usando una struttura ausiliaria (detta DFA dei prefissi viabili) ed usando anche simboli di look-ahead, si può costruire un parser bottom-up (o shift-reduce) deterministico a partire da una grammatica libera G di classe

- **LR(0)**, (il parser risultante non usa simboli di look-ahead – 0);
- **SLR(1)**, (per costruire il parser, si usa implicitamente un simbolo di look-ahead – 1);
- **LR(1)**, (per costruire il parser, si usa esplicitamente, nella costruzione del DFA dei prefissi viabili, un simbolo di look-ahead –1);
- **LALR(1)** (tipo di grammatica usata da YACC, uno strumento automatico che genera parser bottom-up).

Non tutte le grammatiche sono adatte al bottom-up parsing

- Ci possono essere problemi con produzioni del tipo $A \to \epsilon$.
- Assumiamo che la nostra grammatica contenga la produzione $S \to \epsilon$. L'automa PDA bottom-up associato è



  da cui si vede che la riduzione è sempre applicabile.
- Quindi cercare di evitare di usare grammatiche libere con produzioni ϵ quando si vuole usare la tecnica di bottom-up parsing (shift-reduce).